

# **CSC 258: Algorithms for Genomic Data**

## **Syllabus**

### **Course Information**

- CSC 258: Algorithms for Genomic Data
- Smith College Department of Computer Science
- Instructor: Halie Rando
- Semester: Fall 2026
- Class time: WF 1:20 - 2:35
- Location: Bass 002
- Office hours: Monday 1:45–2:45 p.m.
- Second office hour: TBD based on student availability survey

### **Course Description**

This course introduces core algorithms used in bioinformatics, focusing on how biological sequences are transformed into computational representations for large-scale analysis. Students study string algorithms for alignment, k-mer counting, indexing, and genome assembly, emphasizing implementation, efficiency, and design tradeoffs. The course also explores how sequence-derived features support downstream analyses such as dimensionality reduction and association studies. Programming assignments and a final project provide hands-on experience working with real genomic data.

### **Prerequisites**

- CSC 210: Data Structures
- Familiarity with basic programming in Python (or equivalent)
- No prior biology coursework is required

# Course Overview and Goals

Biological sequence data is one of the largest and most computationally demanding forms of modern scientific data. This course examines how genomes are represented, counted, assembled, compared, searched, and indexed using algorithmic techniques.

The course begins with DNA strings, exact and approximate matching, k-mers, and the time and memory costs of different representations. We then connect sequencing reads to genome reconstruction, developing overlap and de Bruijn graph formulations and Eulerian reconstruction while considering the complications introduced by errors, repeats, and uneven coverage.

The second half of the course covers sequence alignment through dynamic programming, computational phylogenetics, and genome indexing with the Burrows–Wheeler Transform and FM-index. The final unit brings these ideas together through the design, implementation, testing, and benchmarking of a genomic data-processing tool.

Throughout the course, we emphasize practical implementation, empirical evaluation, biological and computational correctness, and reasoned tradeoffs among runtime, memory use, scalability, and maintainability.

## Learning Goals

By the end of the course, students will be able to:

- Explain how biological sequences are represented computationally and why representation choices matter.
- Implement and evaluate core string and graph algorithms used in bioinformatics.
- Formulate sequence alignment as an optimization problem and solve it using dynamic programming.
- Select, implement, and evaluate data structures for efficient counting, assembly, and search in genomic datasets.
- Analyze tradeoffs among runtime, memory use, implementation choices, and scalability in genomic data processing.
- Translate biological questions and biological data into appropriate computational formulations.
- Test and benchmark a genomic data-processing tool, use evidence and feedback to identify limitations, and make justified revisions.
- Communicate computational-biology concepts, methods, and results to both technical and broader audiences.

## Textbook

We will use the textbook *Bioinformatics Algorithms: An Active Learning Approach* by Phillip Compeau and Pavel Pevzner. This text emphasizes algorithmic thinking through interactive problem solving and provides clear introductions to core topics such as string alignment, genome assembly, graph algorithms, and indexing.

The book is structured around active exercises and computational challenges that reinforce implementation and design tradeoffs. We will emphasize implementation and computational intuition over formal derivations. Assigned readings will be supplemented with lecture notes and Rosalind problems tailored to the course.

## Grading

| Category                          | Includes                                                        | Weight |
|-----------------------------------|-----------------------------------------------------------------|--------|
| Reading                           | Check-ins and metacognitive reflections                         | 10%    |
| Rosalind                          | Problems and metacognitive reflections                          | 15%    |
| Participation & Engagement        | In-class activities, discussion, guest talks, and course events | 5%     |
| Midterm                           | Checkpoints and final Wikipedia submission                      | 20%    |
| Final Project: Development        | Mini-project checkpoints and code review                        | 20%    |
| Final Project: Initial Submission | Project and explanation/interpretation                          | 15%    |
| Final Project: Iteration          | Benchmarking, feedback, final iteration, and reflection         | 15%    |

Programming work will primarily use Python, though students may use other languages for the final project with instructor approval.

## Midterm Project

For the midterm, each student will improve a computational biology article for the ISCB Wikipedia competition. The project proceeds through topic proposal and instructor approval, registration of the approved topic, analysis of the existing article, a complete draft, peer review, and publication of the final revision on Wikipedia. Although the external competition remains open until May, the final article and accompanying reflection must be submitted for the course by November 13. See the [Midterm page](#) for requirements, resources, and deadlines.

## Final Project

Throughout the semester, students will complete checkpoints on component algorithms that they will need for the final project, including code reviews on September 30 and November 6. Code Review 2 focuses on the graph/reconstruction project. Students will then submit a project plan, a minimum viable implementation, and a first complete version of a substantial genomic data-processing tool. After benchmarking that version in class and receiving feedback, students will revise it and submit a final iteration with a reflection on what changed and why. Across these stages, the project emphasizes algorithm design, correctness, performance evaluation, scalability, and justified implementation decisions. See the [Assignments](#) and [Final Project](#) pages for requirements and deadlines.

# Tentative Course Sequence

- **Meetings 1–2:** Course introduction; Git and GitHub fundamentals
- **Meetings 3–8:** DNA strings; k-mers; counting; representations; algorithmic complexity; k-mer counter design
- **Meetings 9–10:** Approximate matching and selected motif material; sequencing reads, k-mer composition, and the reconstruction problem
- **Meetings 11–13:** Genomic data from sample to biological inference; project work; modern computational genomics at scale
- **Meetings 14–16:** de Bruijn graphs; Eulerian reconstruction; errors, repeats, coverage, and k-mer spectra
- **Meetings 17–19:** Sequence comparison; dynamic programming; global and local alignment
- **Meeting 20:** Computational phylogenetics
- **Meeting 21:** Genome indexing and search; Burrows–Wheeler Transform and FM-index
- **Meetings 22–26:** Final-project architecture, data representations, real-world data, testing, benchmarking, profiling, technical review, and demonstration

The dated [Schedule](#) is the source of truth for class topics and project checkpoints. It may be adjusted in response to Mountain Day, guest availability, or the pace and needs of the class.

## Academically Engaged Time

Students should expect to spend approximately 8–10 hours per week outside of scheduled class meetings. This includes time spent on readings, Rosalind problems, metacognitive reflections, midterm checkpoints, testing and debugging code, and final-project development. Weeks with major project milestones may require additional time.

## Communication & Response Norms

The course website is the source of truth for policies, deadlines, and updates. Please use the channels below to keep questions organized and response times predictable.

### Primary channels

- GitHub Issues (or PRs) are preferred for assignment questions and clarifications.
- Email is for private or time-sensitive matters.
- Coursework may be submitted through GitHub repositories or pull requests, [Smith's JupyterHub](#), or Moodle, as specified for each assignment. Moodle may also be used for feedback.

### Email expectations

- Email is checked once per business day.
- You can expect a response within 24–48 business hours.

- Emails sent outside business hours are treated as arriving the next business day.
- Emails about CSC 258 should use the subject line format CSC 258: <brief description>.

## **How to ask for help effectively**

- Put your full question in one message.
- Include context (assignment name, deadline, and what you tried).
- Avoid sending multiple follow-ups before receiving a reply.
- Emails that do not use the required subject format may receive delayed responses.

## **Office hour cancellations**

If office hours are cancelled, students may attend any remaining office hours that week or use email to ask brief questions. No student will be penalized for missing office hours due to a cancellation.

## **Assessment**

Assessment in this course includes:

- Reading check-ins and metacognitive reflections
- Rosalind problems and metacognitive reflections
- Participation and engagement
- Midterm checkpoints and the final Wikipedia submission
- Final-project development checkpoints and code review
- The initial final-project submission and its explanation and interpretation
- Benchmarking, feedback, iteration, and reflection on the final project

## **Programming Work**

Rosalind problems and final-project work will emphasize algorithm implementation, correctness, explanation, reflection, and benchmarking where appropriate. Deadlines and submission links will be posted on the course website.

## **Regrades**

Regrade requests must include a brief explanation of the grading concern and any relevant evidence. Regrades are handled after grades are returned and may result in a score going up, down, or remaining the same.

## **Extension Policy**

If you need an extension on a deadline, please send me an email with the name of the assignment in the subject line.

## **Attendance and Participation**

Class attendance is required unless you have let the professor know about your absence (e.g., for illness, conference travel, etc.) via email. In-class activities are designed to be completed during the scheduled class time and may be difficult or impossible to replicate independently. Students who are present and participate in class receive full credit for the work. If you have an unexcused absence, you are responsible for making up any missed activities within one week, by coordinating with the instructor or TAs, preferably before the next class.

You can take notes however you want while information is being presented. During some exercises, you will be asked to “clamshell” (close) your computer to work hands-on or to sketch out ideas. If you need accommodations, please reach out to the instructor or to the Accessibility Resources Center (ARC).

## **Course Feedback**

The Fall 2026 course feedback questionnaire will be available from December 9, 2026, through January 5, 2027. We will reserve approximately 15 minutes during the final week of classes for students to complete it. Your thoughtful and constructive feedback is valuable and helps improve the course for future students.

Research indicates that course evaluations can be influenced by conscious or unconscious biases concerning an instructor’s gender, race, ethnicity, and other aspects of identity. When evaluating this course, please reflect on whether the instructor’s identity or your expectations about instructors may be affecting your perceptions. Please focus your feedback on specific aspects of the course—such as its design, learning activities, assignments, communication, and teaching practices—and offer constructive suggestions wherever possible.<sup>12</sup>

## **Academic Integrity**

### **Academic integrity process**

If I have concerns about academic integrity, I am required to meet with the student to share those concerns and then refer the matter to the Academic Integrity Board to manage next steps. I do not adjudicate whether or not a violation occurred. More information is available here: <https://www.smith.edu/your-campus/offices-services/dean-college/academic-integrity-board>.

### **Generative AI**

In this class, generative AI is not allowed in our classroom or TA hours. You should use a code editor that has no generative-AI extensions or features installed or enabled. This means that you should not use AI-enabled editors such as Cursor for coursework. You should not use generative AI on work that is submitted to me or your team members. You should not use generative AI to critique peer code during

code review. If I have concerns that generative AI is being used for the work that is supposed to be representative of your learning process, I am required by Smith policy to report it.

---

1. Östlin, O., Möller, R., Brage, T., & Espersson, M. (2020). Gender bias in students' course evaluations: Risk analysis in five dimensions of previous research and examples at Lund University. [?]
2. LeFebvre, L. E., Carmack, H. J., & Pederson, J. R. (2020). "It's only one negative comment": Women instructors' perceptions of (un)helpful support messages following hurtful course evaluations. *Communication Education*, 69(1), 19–47. [?]